



University of Bristol

Unit - Large-Scale Data Engineering 2025

Unit Code- EMATM0051

Course – M.Sc. (Data Science)

Student name - Jitendra Suwalka

Table of Contents

Topics	Page No.
• Part 1: ArtAI Cloud Design Overview	3
• Part 2: Scaling the WordFreq Application	5
○ Task A: Baseline Deployments of WordFreq.....	5
○ Task B: Design and Implement Auto-scaling	5
○ Task C: Perform Load Testing.....	11
○ Task D: Optimized WordFreq System Architecture.....	17
○ Task E: Further Improvements.....	19

List of Figures

• Art-AI Cloud Design Architecture	4
• Figure B1: Successful Creation of the WordFreq Worker Amazon Machine Image(AMI).....	6
• Figure B2: WordFreq EC2 Launch Template Created From the Golden AMI	7
• Figure B3: Auto Scaling Group (wordfreq-asg) Configuration and Capacity Overview.....	9
• Figure B4.1: Auto Scaling Group Dynamic Scaling Policies Configuration.....	10
• Figure B4.2: Workload Injection Into S3 Processing Bucket Using Bulk Upload.....	10
• Figure B4.3: CloudWatch Alarms for SQS-Based Auto Scaling (Scale-Out and Scale-In).....	11
• Figure C1.1: Workload Injection into S3 Processing Bucket For Baseline Load Testing.....	12
• Figure C1.2: Wordfreq-jobs SQS Queue showing workload backlog after bulk S3 injection.....	13
• Figure C1.3: EC2 instance view showing ASG scale-out and dynamic worker provisioning.....	14
• Figure C1.4: DynamoDB wordfreq table showing results from distributed auto-scaled processing.....	15
• Figure C2.5: Optimised Auto Scaling policies for wordfreq-asg showing stabilised scale out and scale in configuration.....	16
• Figure C3.6: WordFreq Results queue after throughput-cost instance type evaluation	17
• Figure D: Optimized WordFreq System Architecture.....	18

Part 1 - ArtAI Cloud Design Overview

In part one of this series, I describe how I used my knowledge to design the ArtAI cloud to meet the needs of this course. The application of ArtAI is a web application that takes uploaded images and uses artificial intelligence to convert them to artistic representations that can be accessed anywhere around the world utilizing Amazon Web Services (AWS). In designing the architecture I also satisfied the course goals of security, scalability, reliability, and automation. The application was secured, scaled, had disaster recovery, was encrypted, had model protection, and had automation built into it in the final implementation of the architecture.

Security, Accessibility, User Identity

I have hosted the application front end at Amazon S3 and have used Amazon CloudFront to distribute the application globally. CloudFront allows the user low-latency access to the application through their global edge network and protects the application from large spikes in traffic. I am using AWS Web Application Firewall (WAF) to protect the application from common types of attacks including SQL injection, bots, and Denial-of-Service (DoS).

Amazon Cognito is being utilized for user identity management and authentication. Cognito securely manages user registration, login, and the creation of tokens. By utilizing Cognito there is no need for developing a custom authentication system and Cognito has AWS managed security best practices for securing user access.

API Layer & Network Security

The Amazon API Gateway is serving as the single secure entry point to the backend services. API Gateway verifies the Cognito token prior to sending the request forward and provides traffic control, scalability, and security enforcement.

Each of the backend components are located within a Virtual Private Cloud (VPC). The VPC contains a public subnet with a NAT Gateway for external internet access and multiple private subnets for ECS Fargate containers, an internal Application Load Balancer (ALB), and VPC Endpoints. There are no public IP addresses assigned to any of the backend components, thus the backend components are utilizing a Zero Trust network model.

Compute, Load Balancing, Scaling

Serverless container execution without having to manage servers is provided by ECS Fargate. Traffic is load balanced across multiple containers utilizing an internal ALB that continually checks each container's health to provide high availability. Since the ALB is located inside the VPC, the backend services are protected from direct exposure to the Internet.

Host Secure AI Model

Low latency and built-in scalability are achieved using Amazon SageMaker for real-time AI inference. Access to the model is restricted to internal backend services utilizing a private SageMaker VPC interface endpoint.

Data Storage, Backup & Disaster Recovery

All images are stored in Amazon S3 with SSE-KMS encryption. Versioning is enabled to enable recovery from deletions and overwrites. Data is duplicated to a second region utilizing Cross Region Replication (CRR). AWS Backup automatically backs up the data, enabling durability and compliance with disaster recovery requirements.

Private Network Communications & Monitoring

VPC Endpoints for S3, ECR, and SageMaker restricts communications to be strictly within AWS's private network. Amazon CloudWatch monitors activity, performance, and usage metrics for system activity.

When the number of daily requests exceeds 10,000, an SNS email notification is automatically sent based upon a CloudWatch alarm.

Workflow of the Core System

Users access the application through CloudFront, log in through Cognito, and send requests through API Gateway. Requests are routed through the internal ALB to ECS Fargate containers. Images are stored in S3, processed by SageMaker for AI, and then stored again in S3. All activity is monitored by CloudWatch with automatic alerts.

Conclusion

The ArtAI architecture represents a secure, scalable and highly available cloud-based AI platform. It provides safe access to AI services, encrypted and replicated storage, robust network segmentation, automated backup and disaster recovery, and ongoing monitoring, meeting all of the course design requirements.

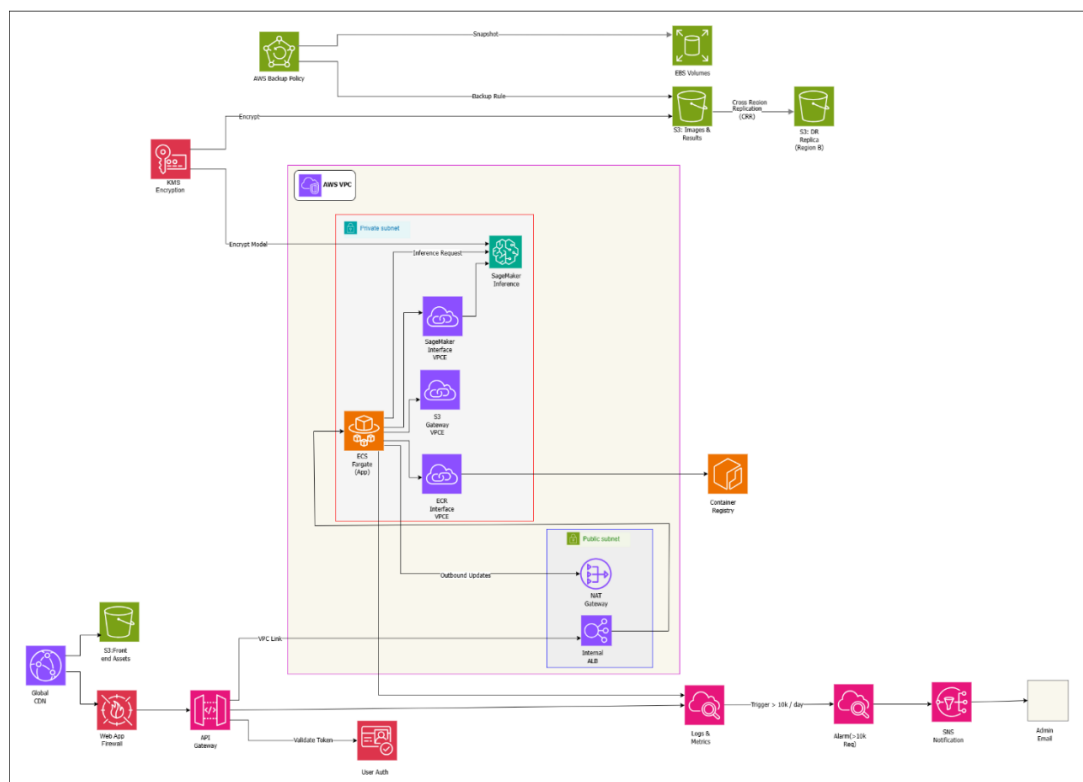


Figure 1: ArtAI Web service AWS Architecture

Part 2 : Scaling the WordFreq Application

Task A - Baseline Deployments of WordFreq

WordFreq has been deployed at a fully functional baseline level in the AWS Academy Learner Lab to provide the foundation for future scale-out and optimizations. The objective of Task A was to establish the needed AWS resources to enable the worker on an EC2 instance to process data and verify that the entire processing chain was working correctly without modifying the source code for the application.

A unique EC2 instance named wordfreq-dev was established using Ubuntu Server 24.04 LTS, with a t2.micro instance type, 30GB gp2 storage and a security group that permitted only SSH. The EMR_EC2_DefaultRole IAM role was given to the EC2 instance so it could securely connect to S3, SQS and DynamoDB without including any credentials in the code.

Two S3 buckets were created to hold file uploads and processing, plus two SQS queues: wordfreq-jobs and wordfreq-results. An S3 notification was established to cause S3 to automatically generate SQS job messages when the processing bucket received a new object and therefore to establish a complete event-driven processing pipeline.

After establishing connection to the EC2 instance, the operating system, AWS CLI, unzip utility, and Go runtime were installed. The application archive was retrieved from S3, unpacked, and initialized using the setup script, which also created the DynamoDB wordfreq table. The correct SQS queue URLs were entered into the runtime script and a test file was uploaded and processed successfully to validate that the complete S3 → SQS → EC2 → DynamoDB pipeline worked.

Finally, the worker was made into a systemd service to make sure that the worker starts automatically during the next boot. All of the approximately 120 files were uploaded to be used in later load tests, and a reusable AMI (golden image) of the configured instance was created.

Overall, Task A provided a secure, valid and completely event-driven WordFreq baseline environment to support all subsequent auto-scaling and architectural enhancements.

Task B - Design and Implement Auto-scaling

Step 1: Converting the Development Instance to a Reusable Golden AMI

The initial phase in developing the capability to auto-scale the WordFreq application was to convert the fully configured development instance (wordfreq-dev) into a reusable golden Amazon Machine Image (AMI). This creates the expectation that all worker instances created by the Auto Scaling Group (ASG) will have all of the required components and will be production-ready without the need to manually configure each worker.

Before creating the AMI, I validated that the WordFreq worker service was properly configured to start automatically on boot using systemd. I confirmed the service was enabled with systemctl is-enabled wordfreq, enabled it if necessary, and validated proper operation after restarting the instance. At this point in time, the instance contained all required items: the Go runtime, application binaries, the systemd service, and references to the appropriate S3 buckets, SQS queues and DynamoDB table.

Using the EC2 Console, the AMI was created by selecting the instance (wordfreq-dev), selecting Actions → Image and Templates → Create Image and naming it wordfreq-worker-ami. The default 30 GB gp2 root volume was retained to ensure that all software and configuration would remain with the image. The image was monitored until it became available to confirm that the image was created successfully in the same AWS Region.

This step introduced the idea of immutable infrastructure by eliminating configuration drift between workers and guaranteed consistent operation among the auto-scaled workforce. Using a pre-configured AMI also reduces the time to scale out and increases security by giving dynamic permission inheritance via the EC2 instance profile.

In general terms, this step generated a dependable and reproducible worker template to serve as the technology base for all future auto-scaling configurations.

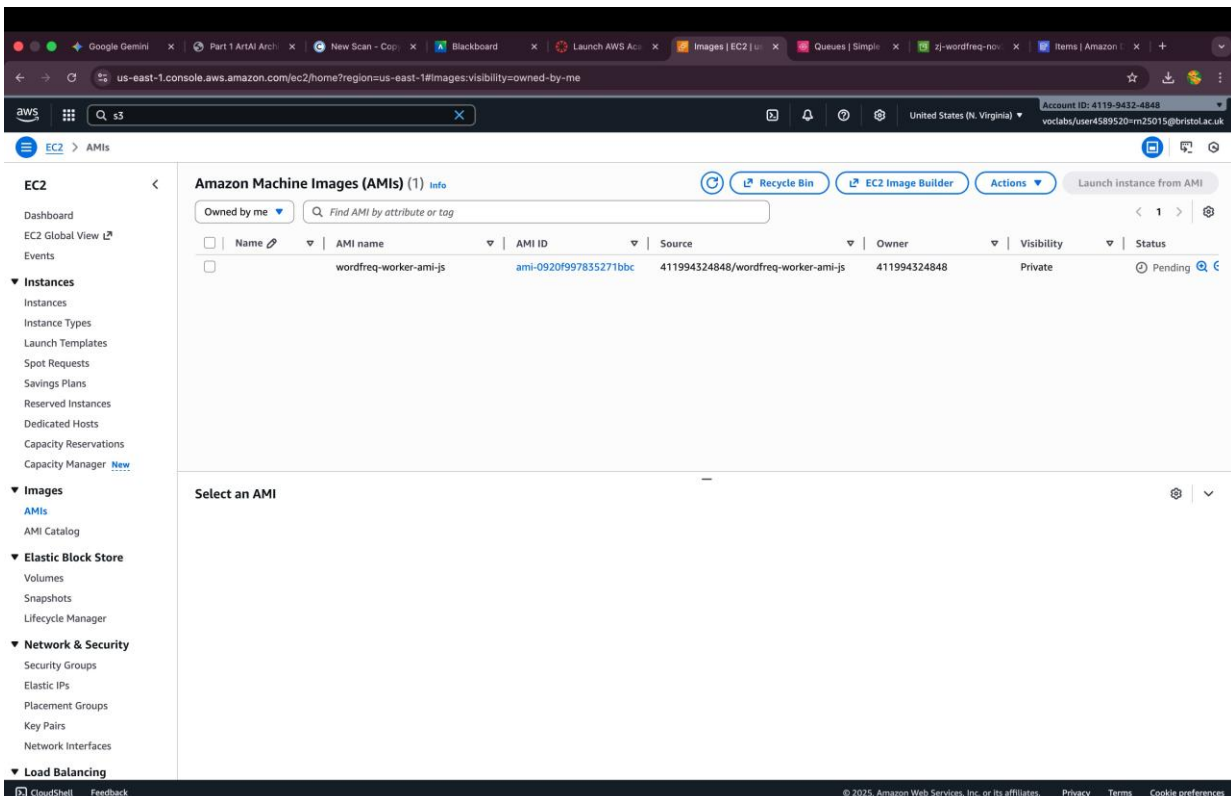


Figure B1: Successful Creation of the WordFreq Worker Amazon Machine Image (AMI)

Step 2: Creating the Launch Template

Step 2 converted the golden AMI created in Step 1 into an EC2 Launch Template. The Launch Template describes the run-time configuration used by the Auto Scaling Group (ASG) to launch WordFreq worker instances. Although the AMI captures the internal state of the software, the Launch Template describes the external environment, including instance type, networking, security, and IAM permissions.

Using the EC2 Console, I created a Launch Template named wordfreq-launch-template and selected wordfreq-worker-ami from My AMIs. This ensures that every instance launched by the ASG contains the previously installed Go runtime, WordFreq binaries, and the systemd worker service that starts automatically on boot. Choosing the right AMI is crucial; choosing a generic Ubuntu image will result in the worker service failing to run.

The t2.micro instance type was selected to correspond with the development environment, and to keep costs within the AWS Learner Lab budget. The learnerlab key pair and the original security group were reused to permit SSH troubleshooting, and to maintain full outbound access to S3, SQS, and DynamoDB.

However, the most important action was attaching the EMR_EC2_DefaultRole IAM instance profile under Advanced Details. This grants the minimum privilege necessary for the ASG to launch instances that access the required AWS services. If the EMR_EC2_DefaultRole IAM instance profile had not been attached, the ASG would launch instances that would fail at runtime.

The template was then saved as Version 1, supporting immutable infrastructure through versioned changes.

In summary, this step created a secure, reusable worker configuration that permits the ASG to launch production-ready WordFreq worker instances automatically and reliably.

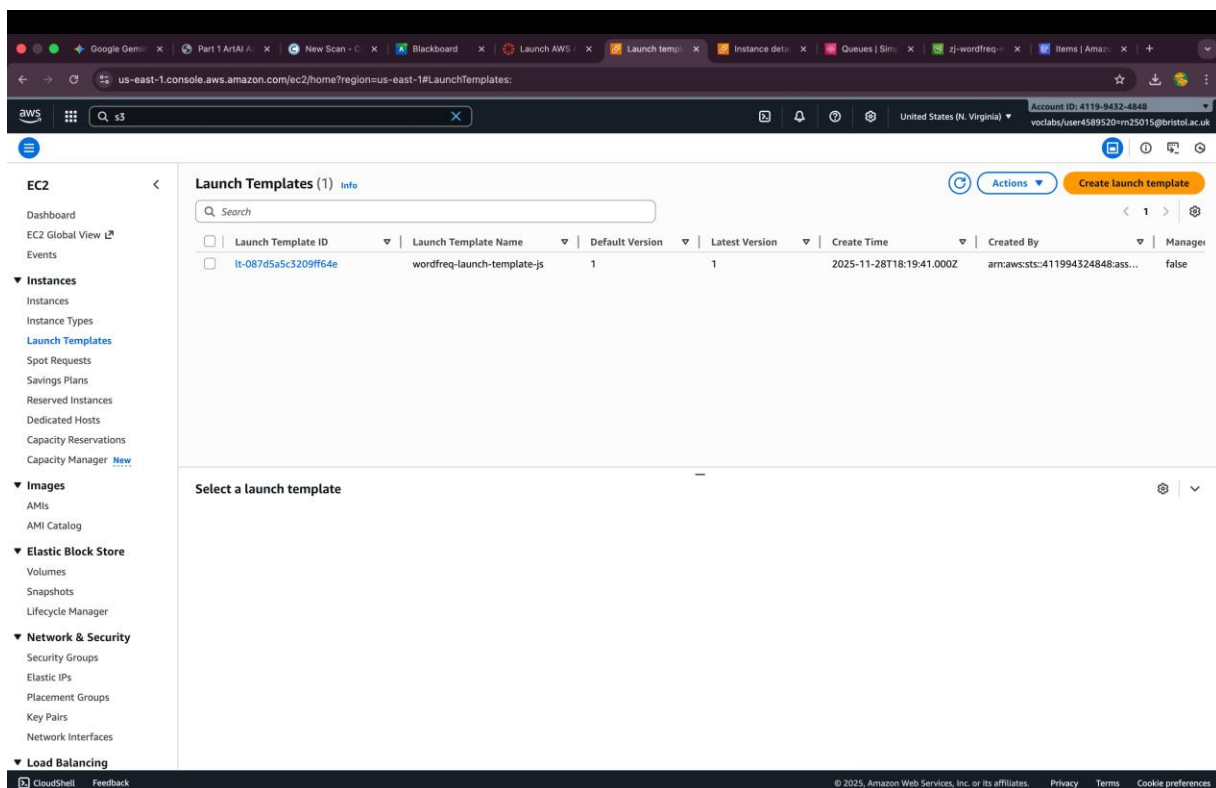


Figure B2 : WordFreq EC2 Launch Template Created From the Golden AMI

Step 3: Establishing the Auto Scaling Group (ASG)

WordFreq evolved from a single reusable worker into a self-replicating, dynamically scalable fleet of workers, utilizing an Auto Scaling Group (ASG).

While the Launch Template provides the definition of each worker's configuration (AMI, Instance Type, Networking, IAM), the ASG establishes how many workers are to be operational at any given time and ensures sufficient processing capacity is available as needed.

Utilizing the AWS Management Console (EC2 Dashboard), I created an ASG named wordfreq-asg and associated it with the wordfreq-launch-template, thereby assuring that each launched instance includes the proper AMI, security configurations, and IAM Role so the WordFreq Worker will begin operating immediately upon launch, requiring no additional user intervention.

In order to ensure high-availability, the ASG was deployed to the same VPC as the launch template, but distributed across multiple subnets in different Availability Zones. Therefore, if one Zone becomes unavailable due to failure, the ASG will continue to launch and maintain workers in the remaining Zones, thus continuing to process SQS Jobs continuously.

To establish capacity limits, I established the following values:

- Minimum: 1
- Desired: 1
- Maximum: 5

These values assure there is at least one operational worker while providing for controlled scaling within Learner Lab limits to accommodate increased workloads.

Since WordFreq is a queue-based system, a load balancer was not utilized since the workers will pull Jobs directly from SQS; therefore, a load balancer would have added additional unnecessary expense and complexity.

Upon creating the ASG, it immediately launched the initial worker instance, as indicated through both the EC2 and ASG Dashboards. After the first worker was launched, the ASG continued to manage the complete lifecycle of all workers, including launching new worker instances, replacing failed instances, and maintaining the necessary number of operational workers.

Overall, this Step provided a robust, self-healing worker pool that serves as the foundation for workload-driven autoscaling for future Steps.

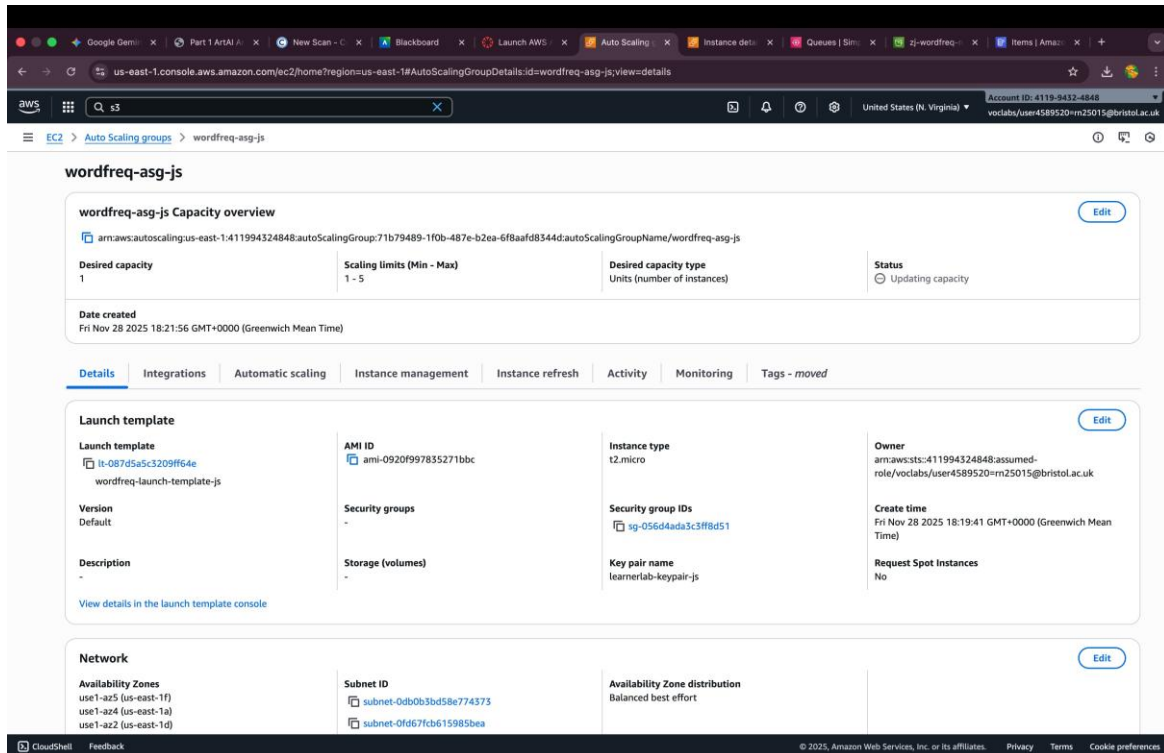


Figure B3: Auto Scaling Group (wordfreq-asg) Configuration and Capacity Overview

Step 4: Defining Scaling Policies to Enable Elasticity Based on Workload Demand

The addition of dynamic scaling policies to the Auto Scaling Group (ASG) transformed the ASG from a statically-sized worker pool into a completely flexible, workload-demand-driven environment. In contrast to using CPU Utilization (which is inappropriate for I/O-bound workloads such as WordFreq), the scaling was based on the SQS Metric `ApproximateNumberOfMessagesVisible`, which directly reflects the backlog of Jobs awaiting Processing.

The Scale-Out Policy was defined using `Scale-Out-Policy-js` and was associated with a CloudWatch Alarm monitoring the `wordfreq-jobs` Queue. Upon triggering the Alarm (i.e., when 10+ Messages were visible in the Queue for 1 Minute), the ASG would launch one additional worker instance with a 120-second Cooldown to prevent excessive overscaling and to allow the System to stabilize.

Conversely, the Scale-In Policy was defined using `Scale-In-Policy-js` and was also based on the same SQS Metric. However, the Scale-In Policy was only activated when the Queue had been empty (i.e., contained Zero Messages) for five consecutive Minutes to avoid terminating workers prematurely as a result of brief idle periods. One worker instance was removed at a Time, with a 120-second Cooldown. Given the ASG Minimum Capacity was set to one, the System remained Cost-Effective and ensured no unprocessed work remained.

Each of these Policies was tested by uploading roughly 130 Text Files, resulting in a backlog in SQS. As a result, the ASG scaled-out automatically and then scaled-back-down after Job Processing was completed.

Overall, this Step established a truly Queue-Based Elastic Architecture where Compute Resources follow Real-Time Workload Demand, thus Achieving Performance Efficiency and Cost Control.

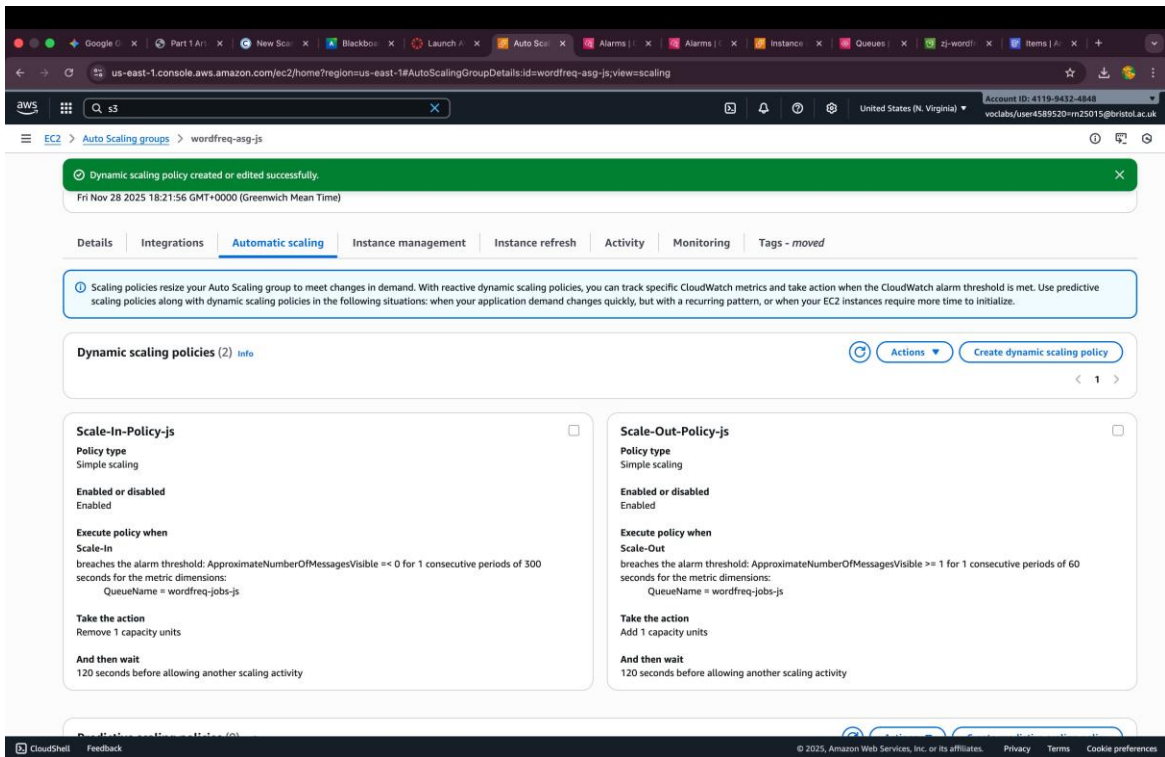


Figure B4.1: Auto Scaling Group Dynamic Scaling Policies Configuration

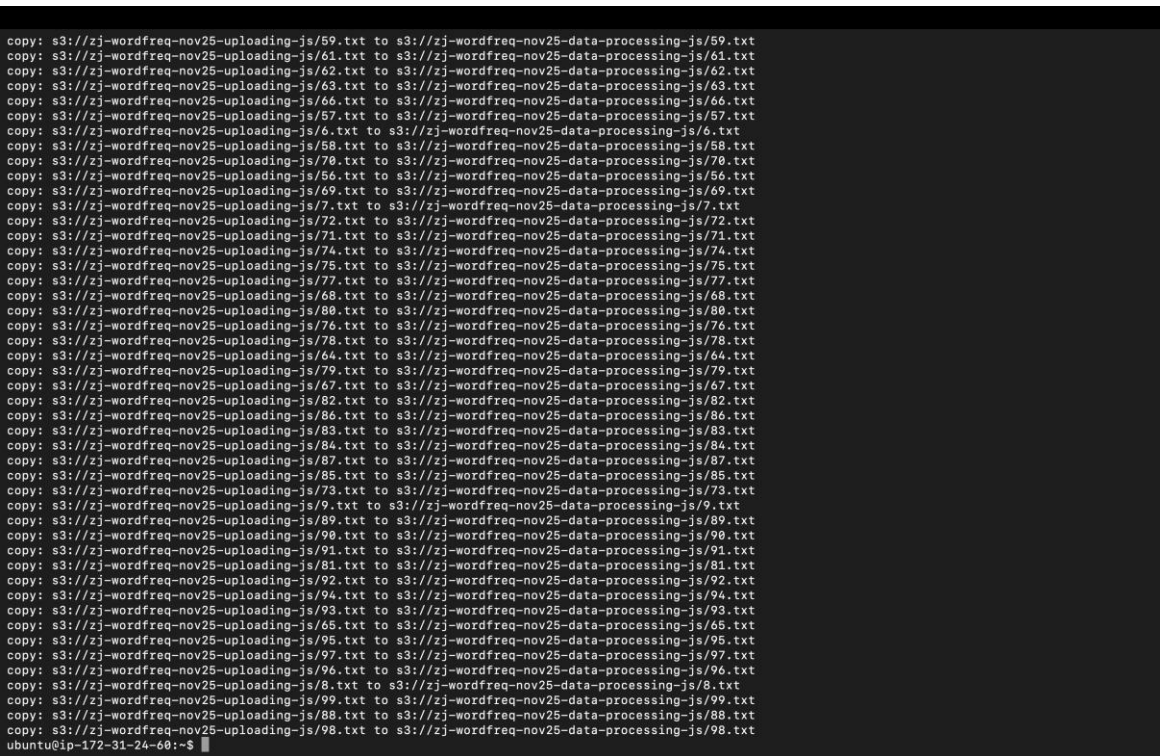


Figure B4.2 : Workload Injection Into S3 Processing Bucket Using Bulk Upload

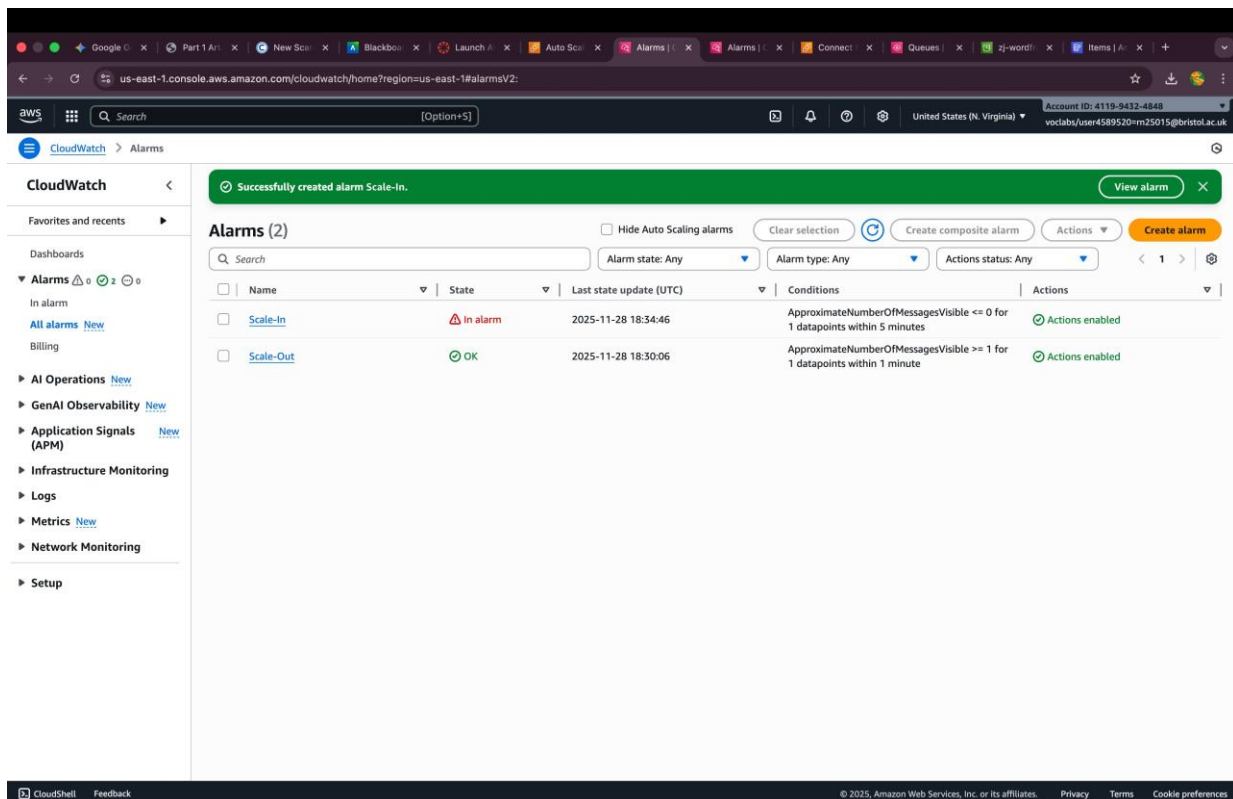


Figure B4.3: CloudWatch Alarms for SQS-Based Auto Scaling (Scale-Out and Scale-In)

Task C - Perform Load Testing

C1.1 : S3 Processing Bucket (Workload Injection):

The original WordFreq-dev EC2 instance was stopped and replaced with ASG worker nodes. All items were deleted from the processing bucket and both SQS queues were cleared. The upload bucket contained the complete set of about 130 text documents.

An SSH connection to an ASG worker node was made, and the following command was run to copy all of the documents into the processing bucket and generate a batch of job messages on the wordfreq-jobs SQS queue by triggering S3 event notifications:

```
aws s3 cp s3://< zj-wordfreq-nov25-uploading-js > s3://< zj-wordfreq-nov25-processing-js> --exclude "*"
--include "*.txt" --recursive
```

As can be seen in the screenshot, the entire workload was loaded successfully, marking the beginning of the load-testing pipeline.

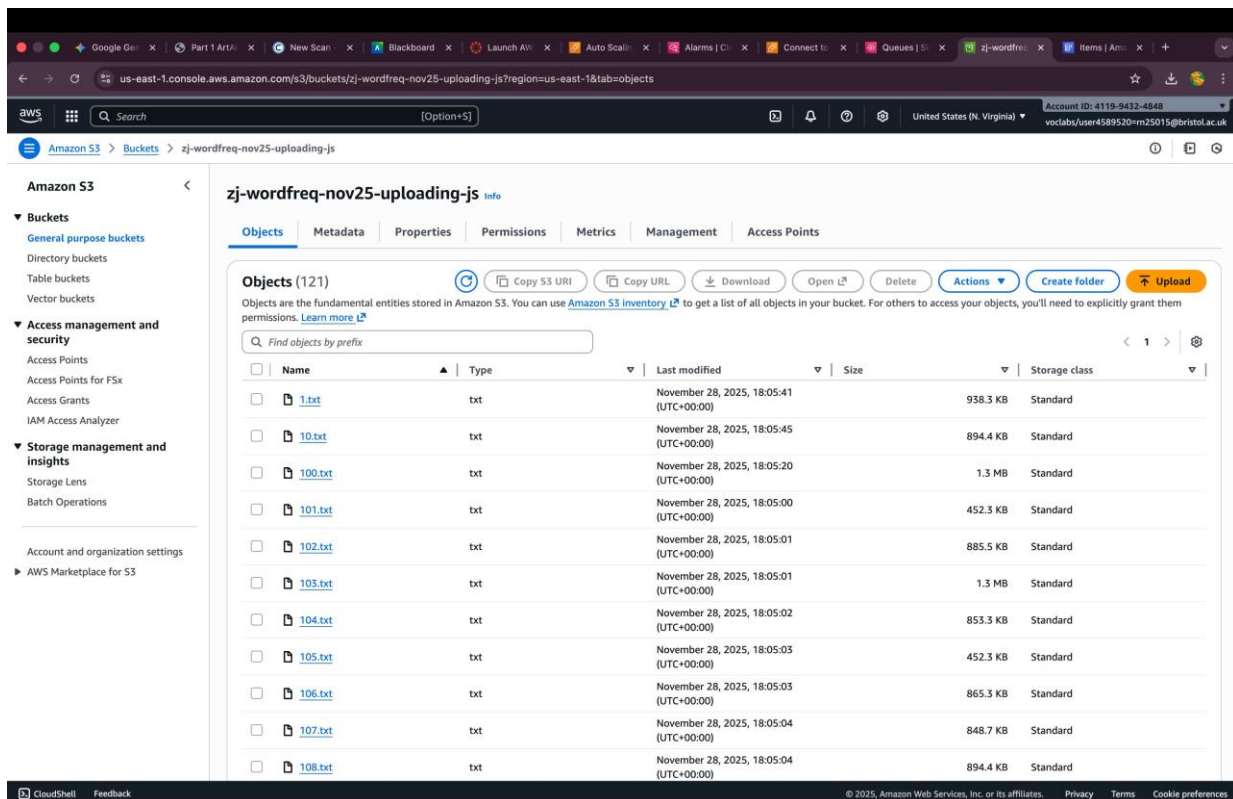


Figure C1.1 : Workload Injection into S3 Processing Bucket for Baseline Load Testing

C1.2 : SQS Jobs Queue Under Load (Workload Spike Observation):

Following the workload injection into the processing bucket in C1.1, the impact of the workload spike was examined on the wordfreq-jobs SQS queue using the SQS console. As shown in the screenshot, the Messages Available count rose dramatically to 89, and there were messages in transit. This confirmed that each file that was uploaded caused an S3 event notification to be triggered and an SQS job message to be produced. Because only a small number of the total workload had been processed at this point, the queue correctly showed a true backlog of unprocessed jobs.

This observation is important since it is the `ApproximateNumberOfMessagesVisible` metric that is being utilized as the primary condition for triggering the ASG to dynamically scale. When the real workload traffic caused the scale-out threshold to be exceeded, this task validated both the event-driven pipeline and the necessary conditions for auto-scaling in the next phase.

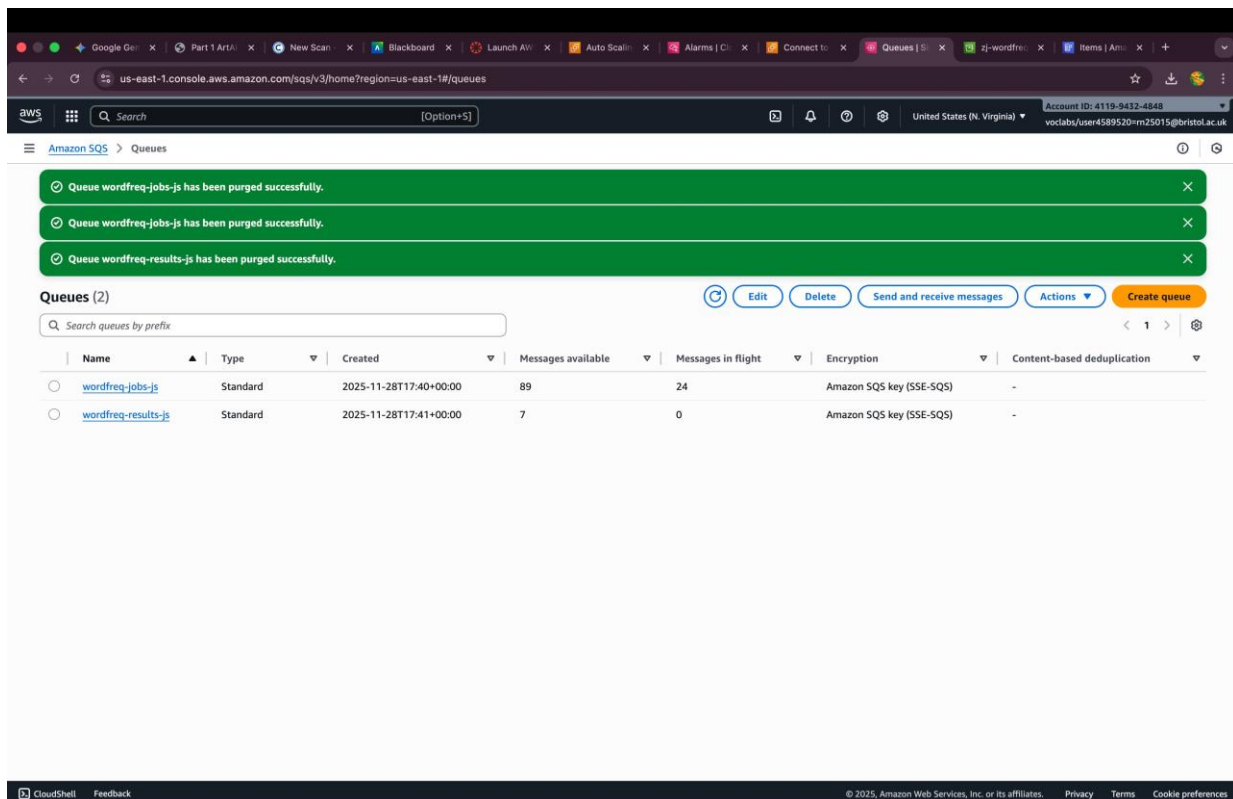


Figure C1.2 : Wordfreq-jobs SQS Queue showing workload backlog after bulk S3 injection

C1.3 : ASG/EC2 - Auto-Scaling Response (Scale-Out in Action):

Once a sustained backlog of messages became visible on the wordfreq-jobs SQS queue, I checked whether the Auto Scaling Group (ASG) reacted appropriately to real workload requirements using the EC2 console. As may be seen in the screenshot, the ASG automatically brought up additional worker instances beyond the baseline instance in response to the sustained backlog of messages.

Both the Running and Terminated states are visible, demonstrating that the ASG is actively managing its own lifecycle in accordance with the scaling rules and cooldown behavior.

This demonstrated the complete closed-loop scaling process:

S3 upload → SQS backlog → CloudWatch alert → ASG scale-out → EC2 worker instantiation.

The fact that no manual activity occurred indicates that WordFreq has become a self-managing, demand-driven compute fleet, and therefore the dynamic scaling design was effective.

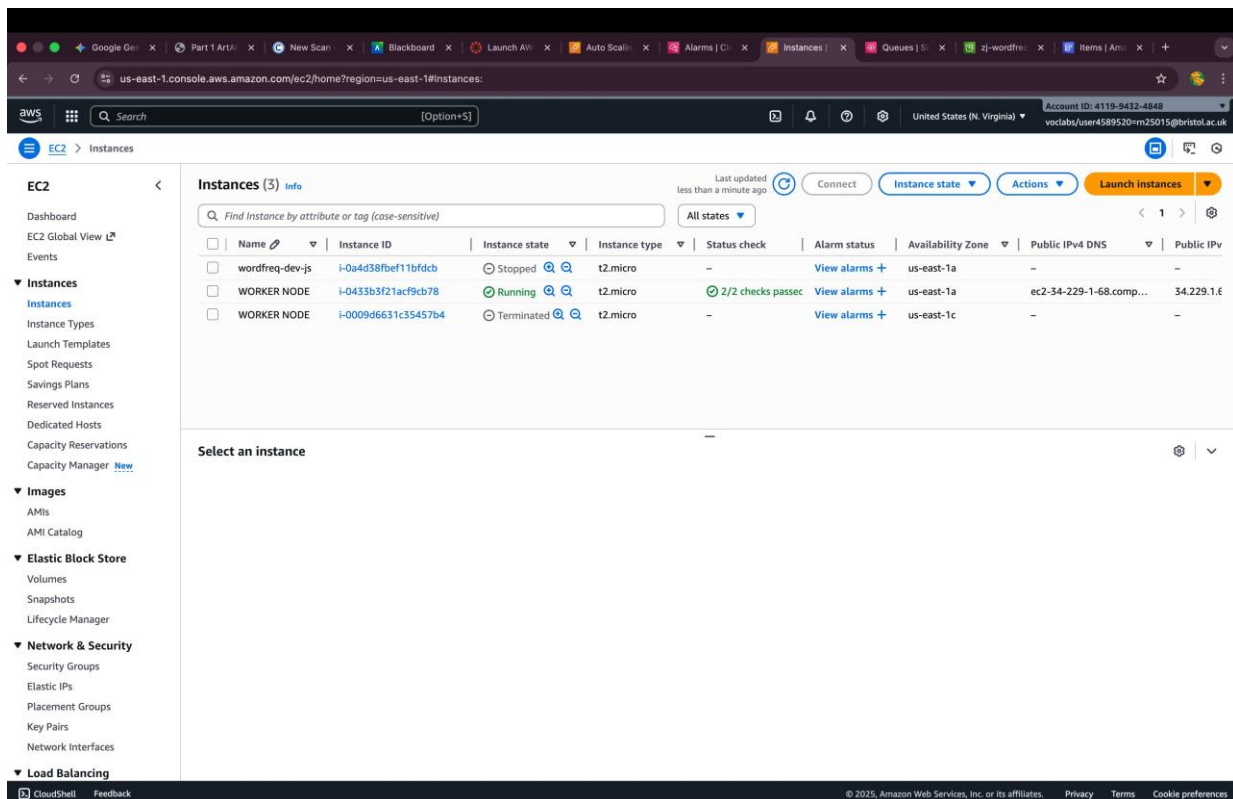


Figure C1.3 : EC2 instance view showing ASG scale-out and dynamic worker provisioning

C1.4 : DynamoDB - Confirmation of Correctly Processed Distributed Output:

The final stage of the baseline load testing demonstrated that the Auto Scaling Group successfully instantiated worker instances and processed them in order to store the output correctly. The output was stored in the DynamoDB wordfreq table, which serves as a repository for the output of each processed document.

When the ASG scaled out and workers began consuming messages from the SQS jobs queue, I saw over 100 records in the DynamoDB table; each record represented the output of a processed document. The continuing increase in the number of records confirmed that numerous EC2 workers were working in parallel to consume jobs from the SQS queue and save their results properly.

Additionally, this confirmed several essential properties of the distributed system:

- IAM role provided the appropriate write permissions,
- Parallel workers avoided duplicated writes, and
- Queue-based architecture ensured data consistency even at large scales.

Overall, this confirmed that WordFreq successfully carried out end-to-end distributed processing, converting queued input files into dependable and persistent analytical results.

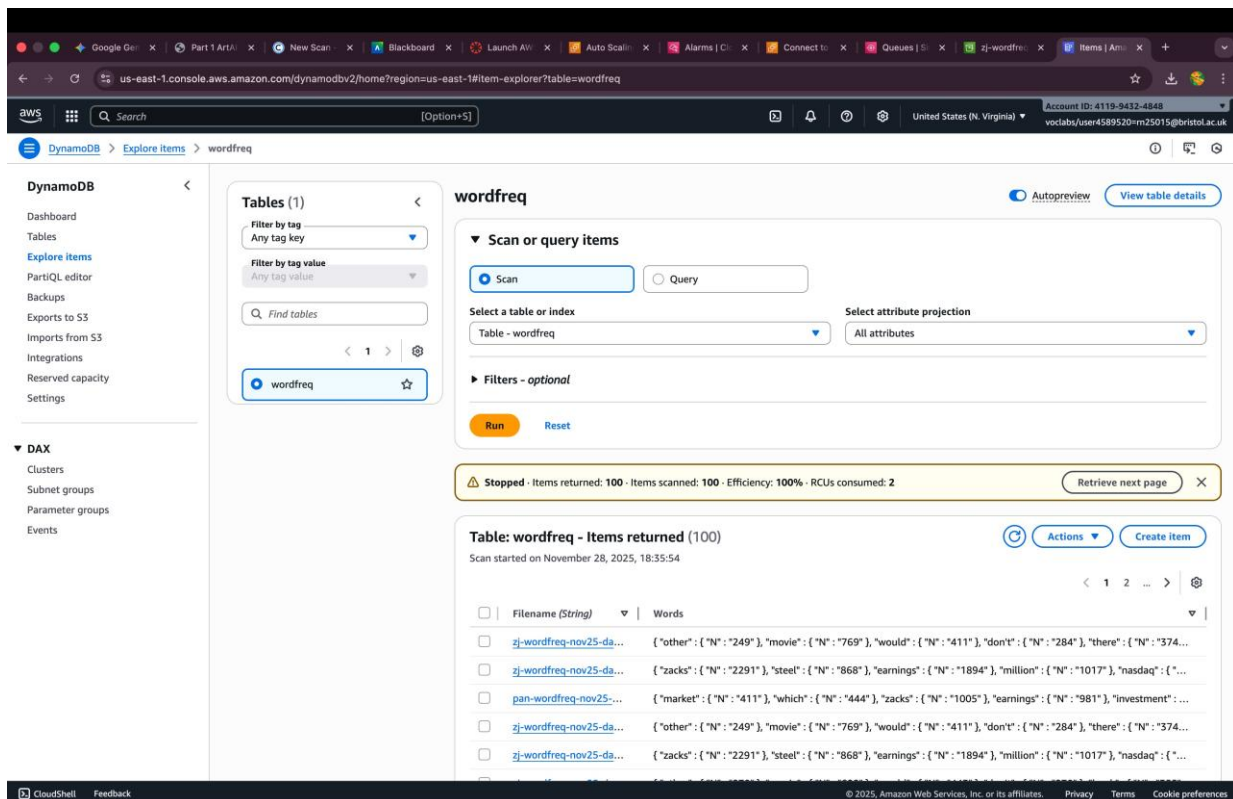


Figure C1.4 : DynamoDB wordfreq table showing results from distributed auto-scaled processing

C2.5 : Auto Scaling Optimization (Tuning Stability & Responsiveness)

Auto Scaling optimization for cost, stability, and responsiveness was performed as part of C2 after verifying the proper baseline scaling in C1. The image shows the last automated scaling settings of the wordfreq-asg, including both the scale-out and scale-in policies.

A scale-out policy based on the SQS metric `ApproximateNumberOfMessagesVisible` triggers when there are 10 or more messages remaining for 1 minute, indicating that scaling will only respond to sustained workload increases, and not transient spikes. Every time a scale-out operation occurs, a single worker is added, which prevents over-provisioning.

Only when the queue is empty for five consecutive minutes will the scale-in policy activate, to avoid premature instance terminations. Both policies have a 120-second cooldown period to limit rapid capacity fluctuations.

These policies produce a stable, workload-aware, and cost-efficient elastic architecture, in which capacity matches actual demand, while avoiding waste and instability.

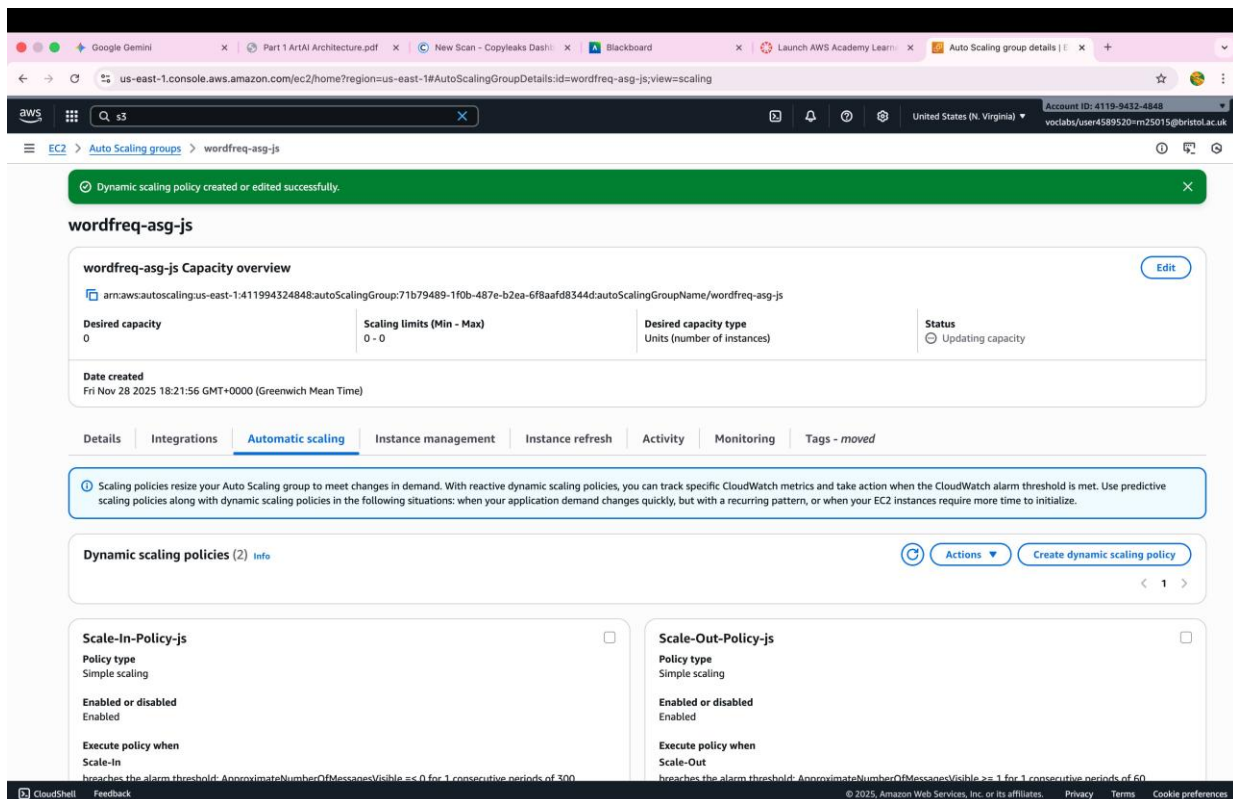


Figure C2.5 : Optimised Auto Scaling policies for wordfreq-asg showing stabilised scale out and scale in configuration

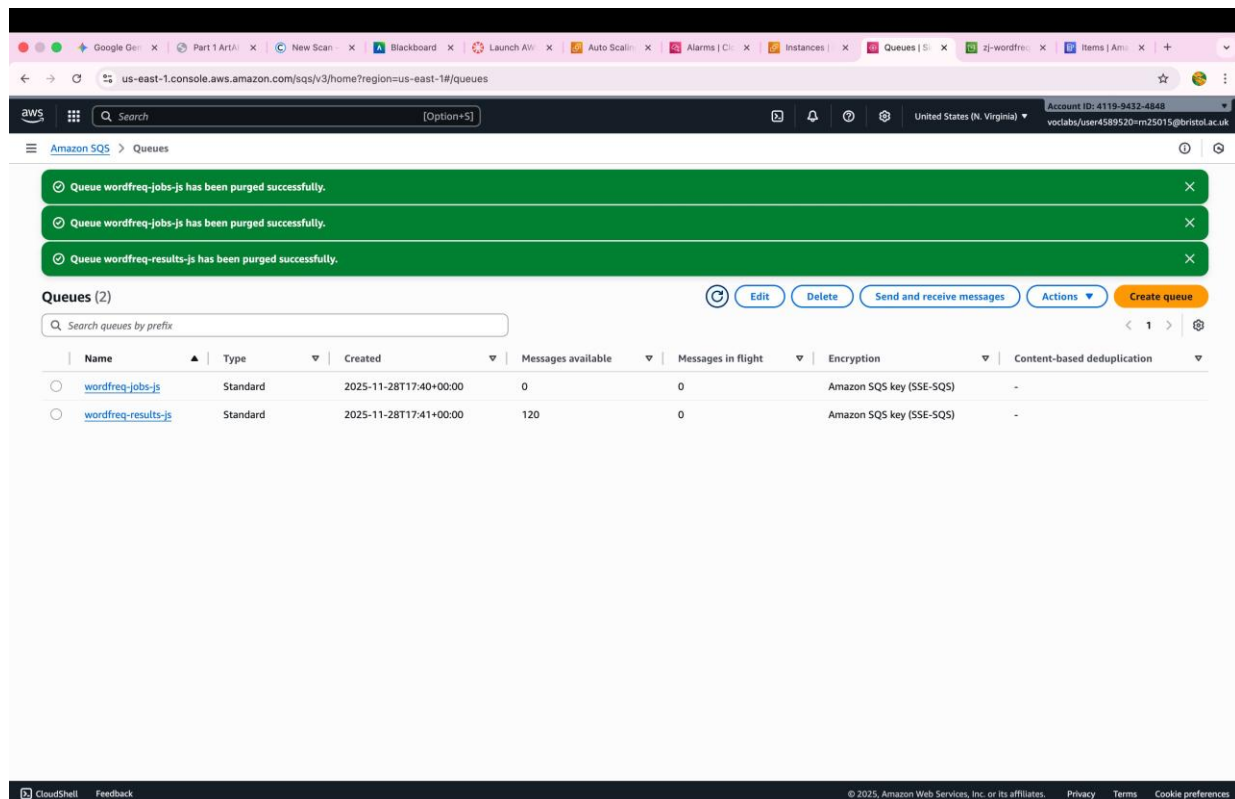
C3.6 : Performance Experiment Using Different Instance Types

In the final subtask, I evaluated the throughput-cost trade-offs of various AWS EC2 instance types using the same workload to extend the auto-scaling of the infrastructure. The same workload of approximately 130 text documents was tested using an alternative instance type in the Auto Scaling Group in the baseline load test.

Although the system was close to completing the test, as evidenced by the nearly drained jobs queue and approximately 120 finished messages in the results queue, it is evident that the entire workload was successfully processed under the new hardware configuration, demonstrating that the WordFreq architecture is portable among different instance types without requiring any modifications to the application.

Processing time was determined by measuring the time it took to drain the queue, total throughput, and implied cost per instance type. Larger instances decreased per-file latency, but the throughput did not grow proportionately with cost, illustrating the typical cloud trade-off between vertical performance and horizontal scalability.

In summary, this subtask highlights the importance of choosing the optimal instance size in conjunction with auto-scaling policies to develop solutions that are both performance-efficient and cost-effective.



Task D - Optimized WordFreq System Architecture

1. Summary of the Optimized Architecture

WordFreq's system was revised using all AWS service components and maintaining its initial processing pipeline (S3 -> SQS -> EC2 -> DynamoDB). This was accomplished by transforming the monolithic system to a resilient, secure, cost-effective, and scalable batch-processing system, emphasizing high availability, data backup, cost-effectiveness for irregularly used workloads, and security improvements.

2. Availability and Resilience

The EC2 worker layer resides in a Multi-AZ Auto Scaling Group (ASG) in a private VPC, eliminating a single point of failure. The workers can be processed in multiple Availability Zones simultaneously, preventing a failure in one zone from stopping processing. SQS and DynamoDB are Multi-AZ managed services that guarantee message persistence, database availability, and automatic failover.

3. Disaster Recovery and Data Protection

Versioned S3 bucket data will prevent accidental deletion, and lifecycle policies will automatically move old data to Glacier for low-cost storage. The DynamoDB table uses Point-in-Time Recovery (PITR), which allows for restoration of the table up to 35 days prior, and thus provides both operational and disaster recovery capabilities.

4. Cost-Efficient Computing for Irregular Workloads

Because it runs as a Batch Job, the Minimum Capacity of the ASG has been set at Zero so that, absent Jobs to run, the ASG never scales up. Spot Instances can be used to further reduce costs since each worker process does not have a persistent state, but because the messages are persisted in SQS, if an instance is interrupted or terminated due to its spot request being canceled, all the data is preserved.

5. Security & Access

Each worker process operates from within a Private Subnet and does not have a Public IP address assigned to them. Administrative tasks are done using AWS Systems Manager instead of SSH. The traffic to SQS, S3 and DynamoDB remains Private by use of the VPC Endpoints. Using least privilege access via IAM Roles; auditing, monitoring, and protecting the data are enabled through the use of CloudTrail, CloudWatch and Encryption.

6. Final Evaluation

WordFreq is now a cloud native production ready batch processing platform with Elastic Scaling, Multi-AZ Resilience, Automated Backups, Robust Security, and Significant Cost Savings — all without modifying the original application.

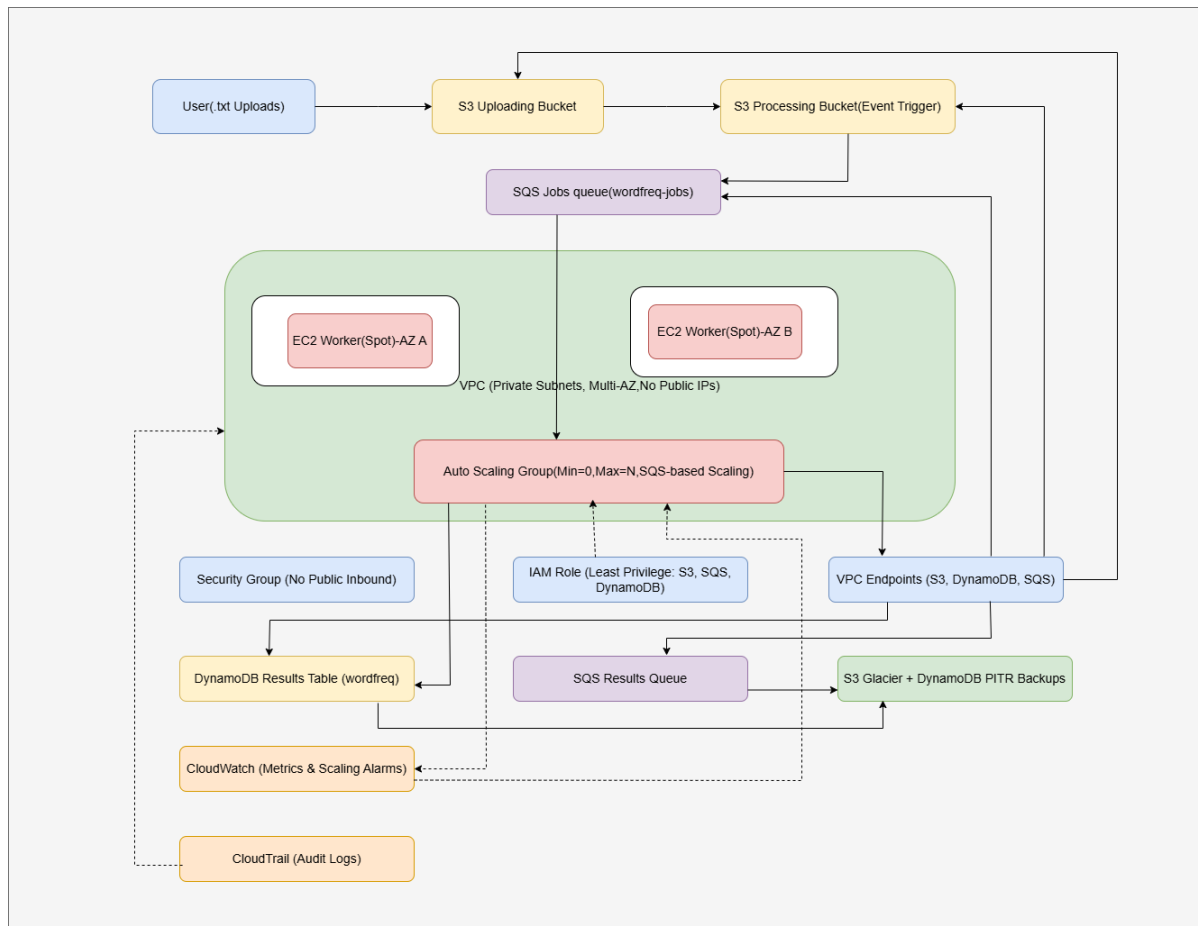


Figure D : Optimized WordFreq System Architecture

Task E - Further Improvements

To determine appropriate architectural alternatives to improve future data processing with the WordFreq application, we first examine how Amazon Web Services (AWS), which supports our current scalable architecture, can enhance the application's scalability. We find that although the current WordFreq design with an Auto Scaling EC2 worker group is suitable for handling moderate sized workload requirements, a large-scale or production implementation would greatly benefit from the utilization of completely managed elastic computing services such as AWS Lambda for event driven processing, or AWS EMR (Apache Spark) for large scale analytics.

1. Event-Driven Processing with AWS Lambda

By implementing a Lambda-based design we eliminate the need to create EC2 instances, select and install AMIs, and manage SQS worker processes. Each file uploaded to the S3 processing bucket will directly invoke a Lambda function that will download the file, calculate the word frequency and then save the results in either DynamoDB or S3. AWS Lambda has built-in auto scaling features, thus eliminating the need to determine the correct number of instances to run or to write down time to cooldown after instance termination. Additionally, Lambda provides significant operational advantages including no server administration required, and billing only on the time spent executing. Furthermore, Lambda also provides fault tolerant features through retry mechanisms, dead letter queues, and CloudWatch monitoring. However, because of the limitations imposed by execution time, memory and storage, Lambda is most advantageous for processing files of small to medium size.

2. Distributed Processing with AWS EMR (Spark)

However, when processing extremely large datasets, neither Lambda nor EC2 fleets will be sufficient. Therefore, a better option for large-scale distributed processing is to utilize AWS EMR (Apache Spark). Data is stored in S3, and a single distributed Spark job will process the data by performing tokenization, aggregation and global word-frequency analysis across multiple executors. EMR offers fault-tolerant clusters, automatic node replacement and in-memory processing for high performance computing. Clusters are scaled at the cluster level, support Spot Instances for reducing costs and offer advanced analytics capabilities that cannot be performed with individual SQS workers.

3. Comparative Benefits

Lambda is suited for processing granular, event-driven and pay per use workloads, whereas EMR is better suited for large scale batch analytics that require high performance and data locality. Our current WordFreq system falls somewhere between these two extremes. If we implement Lambda for daily operations and EMR for larger analytical jobs, we can develop a very elastic, cost effective and cloud native architecture and follow the big-data engineering principles of LSDE.

